

Protokoll: Solver für stetige Funktionen

Nullstellenberechnung mit Bisektion und Regula falsi

Name	Piotr Opalko
Klasse	3BHWII
Schuljahr	2025/2026
Gegenstand	Softwareentwicklung und Projektmanagement
Programm-Titel	Solver für stetige Funktionen
Abgabe	ZIP-Verzeichnis mit Protokoll, PDF und Python-Code

Inhaltsverzeichnis

1. Projektüberblick
2. Ausführung des Programms
3. Aufgabe 5 - Bisektionsmethode
4. Aufgabe 6 - Regula falsi
5. Aufgabe 7 - Grafische Darstellung
6. Aufgabe 8 - Polynom und Genauigkeitstests
7. Aufgabe 9 - Leitung / Kettenlinie
8. Softwaretests und Ergebnisinterpretation

1. Projektüberblick

In diesem Projekt wurde ein Solver für stetige Funktionen mit einer Variablen x erstellt. Ziel ist es, Nullstellen numerisch zu bestimmen. Als erstes Verfahren wird die Intervallhalbierung (Bisektion) verwendet. Als alternative Methode wird Regula falsi umgesetzt. Zusätzlich werden die Ergebnisse getestet und grafisch mit matplotlib dargestellt.

Der Code ist so aufgebaut, dass jede Aufgabe einzeln über ein Startmenü getestet werden kann. Es muss dafür nichts im Code geändert werden.

2. Ausführung des Programms

Das Programm wird in PowerShell oder Terminal wie folgt gestartet:

```
python solver_projekt.py
```

Danach erscheint ein Menü mit den einzelnen Aufgaben:

```
1 = Aufgabe 5: Bisektion testen
2 = Aufgabe 6: Regula falsi testen
3 = Aufgabe 7: Grafische Darstellung
4 = Aufgabe 8: Polynom testen
5 = Aufgabe 9: Leitung berechnen
6 = Eigene Funktion testen
0 = Programm beenden
```

Für Aufgabe 7 wird matplotlib benötigt. Installation bei Bedarf:

```
pip install matplotlib
```

3. Aufgabe 5 - Bisektionsmethode

Bei der Bisektionsmethode wird ein Intervall $[a, b]$ verwendet, in dem ein Vorzeichenwechsel vorliegt. Der Mittelpunkt c wird mit $c = (a + b) / 2$ berechnet. Danach wird entschieden, in welcher Hälfte die Nullstelle liegt. Dieser Vorgang wird wiederholt, bis die gewünschte Genauigkeit erreicht ist.

Getestet wurde mit der Wurzelfunktion $x^2 - n$ für $n = 25, 81$ und 144 .

n	numerische Loesung	analytische Loesung	Iterationen
25	5.0000000047	5	30
81	9.0000000010	9	33
144	11.9999999972	12	34

Die numerischen Ergebnisse stimmen mit den analytischen Lösungen sehr gut überein.

Screenshot / Codeausschnitt:

```

11
12 def bisektion(
13     funktion: str,
14     a: float,
15     b: float,
16     tol: float = 1e-7,
17     max_iter: int = 100
18 ) -> tuple[float, int] | None:
19     """
20     Berechnet eine Nullstelle mit der Bisektionsmethode.
21     Gibt die Nullstelle und die Anzahl der Iterationen zurück.
22     """
23     try:
24         # Prüfen, ob ein Vorzeichenwechsel im Intervall liegt
25         if f(a, funktion) * f(b, funktion) >= 0:
26             print("Fehler: Kein Vorzeichenwechsel im Intervall [a, b].")
27             return None
28
29         # Intervall wird iterativ halbiert
30         for i in range(max_iter):
31             c = (a + b) / 2
32             fc = f(c, funktion)
33
34             # Wenn f(c) nahe genug bei 0 ist, wird c zurückgegeben
35             if abs(fc) < tol:
36                 return c, i + 1
37
38             # Intervallseite auswählen, wo der Vorzeichenwechsel liegt
39             if f(a, funktion) * fc < 0:
40                 b = c
41             else:
42                 a = c
43
44         # Falls max_iter erreicht wird
45         return (a + b) / 2, max_iter

```

4. Aufgabe 6 - Regula falsi

Als alternative Methode wurde Regula falsi programmiert. Diese Methode arbeitet ebenfalls mit einem Intervall $[a, b]$ und einem Vorzeichenwechsel. Im Unterschied zur Bisektion wird der neue Punkt c nicht als Mittelpunkt berechnet, sondern durch die Sekante zwischen den Punkten $(a, f(a))$ und $(b, f(b))$.

Formel:

$$c = (a * f(b) - b * f(a)) / (f(b) - f(a))$$

Auch hier wurden die Werte $n = 25, 81$ und 144 getestet.

n	numerische Loesung	analytische Loesung	Iterationen
25	4.9999999930	5	52
81	8.9999999954	9	99
144	11.9999986660	12	100

Die Ergebnisse nähern sich ebenfalls den analytischen Lösungen. Die Anzahl der Iterationen kann je nach Intervall größer sein als bei der Bisektion.

Screenshot / Codeausschnitt:

```

def regula_falsi(
    funktion: str,
    a: float,
    b: float,
    tol: float = 1e-7,
    max_iter: int = 100
) -> float | None:
    """
    Berechnet eine Nullstelle mit der Regula-falsi-Methode.

    funktion: Funktion als Text, z.B. "x**2 - 25"
    a: linker Startwert des Intervalls
    b: rechter Startwert des Intervalls
    tol: gewünschte Genauigkeit
    max_iter: maximale Anzahl an Iterationen
    """
    try:
        # Zuerst wird geprüft, ob zwischen a und b ein Vorzeichenwechsel liegt.
        # Ohne Vorzeichenwechsel kann die Methode keine sichere Nullstelle finden.
        if f(a, funktion) * f(b, funktion) >= 0:
            print("Fehler: Kein Vorzeichenwechsel im Intervall [a, b].")
            return None

        # Startwert für c, damit c später sicher existiert.
        c = a

        # Die Schleife wiederholt das Verfahren maximal max_iter-mal.
        for i in range(max_iter):
            # Funktionswerte an den Intervallgrenzen berechnen.
            fa = f(a, funktion)
            fb = f(b, funktion)

            # Regula-falsi-Formel:
            # c ist der Schnittpunkt der Sekante mit der x-Achse.
            c = (a * fb - b * fa) / (fb - fa)

            # Funktionswert an der neuen Stelle c berechnen.
            fc = f(c, funktion)

            # Aktuellen Iterationsschritt ausgeben.
            print(f"Iteration {i + 1}: x = {c}, f(x) = {fc}")

            # Wenn f(c) nahe genug bei 0 liegt,
            # wurde die Nullstelle genau genug gefunden.
            if abs(fc) < tol:
                return c

            # Nun wird entschieden, welche Intervallseite ersetzt wird.
            # Die Nullstelle liegt immer dort, wo ein Vorzeichenwechsel ist.
            if fa * fc < 0:
                b = c
            else:
                a = c

        # Falls die maximale Iterationszahl erreicht wurde,
        # wird die letzte Näherung zurückgegeben.
        return c

    except ValueError as error:
        print(error)
        return None

```

5. Aufgabe 7 - Grafische Darstellung

Für die grafische Aufbereitung werden pro Iteration die aktuelle Lösung x und die aktuelle Genauigkeit $|f(x)|$ gespeichert. Danach werden zwei Diagramme mit matplotlib erstellt. Das erste Diagramm zeigt die Genauigkeit je Iterationsschritt. Das zweite Diagramm zeigt die aktuelle Lösung x je Iterationsschritt.

Im Code wird mit `plt.pause()` eine einfache Animation erstellt. Dadurch werden die Diagramme nach jedem Iterationsschritt aktualisiert.

```
def bisektion_verlauf(
    funktion: str,
    a: float,
    b: float,
    tol: float = 1e-7,
    max_iter: int = 50
) -> tuple[list[int], list[float], list[float]]:
    """Speichert den Verlauf der Bisektionsmethode."""
    iterationen = []
    loesungen = []
    genauigkeiten = []

    if f(a, funktion) * f(b, funktion) >= 0:
        raise ValueError("Kein Vorzeichenwechsel im Intervall.")

    for i in range(max_iter):
        c = (a + b) / 2
        fc = f(c, funktion)

        iterationen.append(i + 1)
        loesungen.append(c)
        genauigkeiten.append(abs(fc))

        if abs(fc) < tol:
            break

        if f(a, funktion) * fc < 0:
            b = c
        else:
            a = c

    return iterationen, loesungen, genauigkeiten
```

Screenshot / Codeausschnitt:

```
def regula_falsi_verlauf(
    funktion: str,
    a: float,
    b: float,
    tol: float = 1e-7,
    max_iter: int = 50
) -> tuple[list[int], list[float], list[float]]:
    """Speichert den Verlauf der Regula-falsi-Methode."""
    iterationen = []
    loesungen = []
    genauigkeiten = []

    if f(a, funktion) * f(b, funktion) >= 0:
        raise ValueError("Kein Vorzeichenwechsel im Intervall.")

    for i in range(max_iter):
        fa = f(a, funktion)
        fb = f(b, funktion)

        c = (a * fb - b * fa) / (fb - fa)
        fc = f(c, funktion)

        iterationen.append(i + 1)
        loesungen.append(c)
        genauigkeiten.append(abs(fc))

        if abs(fc) < tol:
            break

        if fa * fc < 0:
            b = c
        else:
            a = c

    return iterationen, loesungen, genauigkeiten
```

```

def zeichne_diagramme(
    funktion: str,
    a: float,
    b: float
) -> None:
    """Zeichnet die Genauigkeit und die aktuelle Lösung."""
    try:
        it_b, x_b, g_b = bisektion_verlauf(funktion, a, b)
        it_r, x_r, g_r = regula_falsi_verlauf(funktion, a, b)

        fig, axes = plt.subplots(2, 1, figsize=(8, 8))

        # Diagramm 1: Genauigkeit
        axes[0].plot(it_b, g_b, marker="o", label="Bisektion")
        axes[0].plot(it_r, g_r, marker="o", label="Regula falsi")
        axes[0].set_title("Aktuelle Genauigkeit je Iterationsschritt")
        axes[0].set_xlabel("Iteration")
        axes[0].set_ylabel("|f(x)|")
        axes[0].legend()
        axes[0].grid(True)

        # Diagramm 2: aktuelle Lösung x
        axes[1].plot(it_b, x_b, marker="o", label="Bisektion")
        axes[1].plot(it_r, x_r, marker="o", label="Regula falsi")
        axes[1].set_title("Aktuelle Lösung x je Iterationsschritt")
        axes[1].set_xlabel("Iteration")
        axes[1].set_ylabel("x")
        axes[1].legend()
        axes[1].grid(True)

        plt.tight_layout()
        plt.show()

```

6. Aufgabe 8 - Polynom und Genauigkeitstests

Das Polynom lautet:

$$P_4(x) = 2x + x^2 + 3x^3 - x^4$$

Da die gesuchte Nullstelle bei ungefähr $x = 3,4567$ liegt, wurde das Intervall $[3, 4]$ gewählt. Dieses Intervall ist passend, weil die gesuchte Nullstelle darin liegt und ein Vorzeichenwechsel vorhanden ist.

Genauigkeit epsilon	Nullstelle	f(x)	Iterationen
10 ⁻²	3.45654296875	0.0066000213	11
10 ⁻⁸	3.4566783430054784	1.8837e-09	30

Interpretation: Für epsilon = 10⁻⁸ werden deutlich mehr Iterationsschritte benötigt als für epsilon = 10⁻². Das ist logisch, weil eine kleinere Toleranz eine genauere Näherung verlangt.

Screenshot / Codeausschnitt:

```

def aufgabe8() -> None:
    """
    Aufgabe 8:
    Test des Polynoms  $P_4(x) = 2x + x^2 + 3x^3 - x^4$ .
    """
    funktion = "2*x + x**2 + 3*x**3 - x**4"

    # Passendes Intervall, weil 3,4567 zwischen 3 und 4 liegt
    a = 3
    b = 4

    print("\nAufgabe 8: Polynom-Test")
    print("Funktion: 2*x + x**2 + 3*x**3 - x**4")
    print("Intervall: [3, 4]")

    # Test mit Genauigkeit  $10^{-2}$ 
    ergebnis_1 = bisektion(funktion, a, b, tol=1e-2)

    if ergebnis_1 is not None:
        loesung_1, iterationen_1 = ergebnis_1
        print("\nGenauigkeit  $\varepsilon = 10^{-2}$ ")
        print(f"Nullstelle: {loesung_1}")
        print(f"Iterationen: {iterationen_1}")
        print(f"Kontrolle: f(x) = {f(loesung_1, funktion)}")

    # Test mit Genauigkeit  $10^{-8}$ 
    ergebnis_2 = bisektion(funktion, a, b, tol=1e-8)

    if ergebnis_2 is not None:
        loesung_2, iterationen_2 = ergebnis_2
        print("\nGenauigkeit  $\varepsilon = 10^{-8}$ ")
        print(f"Nullstelle: {loesung_2}")
        print(f"Iterationen: {iterationen_2}")
        print(f"Kontrolle: f(x) = {f(loesung_2, funktion)}")

    print("\nInterpretation:")
    print("Je kleiner  $\varepsilon$  ist, desto genauer ist die Lösung.")
    print("Dafür werden mehr Iterationen benötigt.")

```

7. Aufgabe 9 - Leitung / Kettenlinie

Eine elektrische Leitung wird zwischen zwei gleich hohen Masten mit 100 m Abstand gespannt. Der maximale Durchhang in der Mitte beträgt 10 m. Zuerst muss der Krümmungsradius a bestimmt werden.

Aus der Randbedingung $y(50) = y_0 + 10$ ergibt sich:

$$a * \cosh(50 / a) - a = 10$$

Als Nullstellenproblem wird daraus:

$$x * \mathit{math}.\cosh(50 / x) - x - 10 = 0$$

Dabei steht x im Programm für den gesuchten Krümmungsradius a . Als Intervall wurde $[100, 200]$ gewählt.

Nach der Berechnung von a wird die Länge mit folgender Formel berechnet:

$$l = 2 * a * \sinh(w / (2 * a))$$

Methode	Kruemmungsradius a	Iterationen	Leitungslaenge
Bisektion	126.6324359924	28	102.6186868149 m
Regula falsi	126.6324360825	14	102.6186868111 m

Die Leitung ist gerundet etwa 102,62 m lang.

Screenshot / Codeausschnitt:

8. Softwaretests und Ergebnisinterpretation

Der Code wurde so vorbereitet, dass alle Aufgaben einzeln über das Menü getestet werden können. Es ist kein Eingriff in den Code notwendig. Für die Tests wurden bekannte analytische Ergebnisse verwendet, zum Beispiel $\sqrt{25} = 5$, $\sqrt{81} = 9$ und $\sqrt{144} = 12$.

```
def aufgabe9() -> None:
    """
    Aufgabe 9:
    Berechnet die Länge einer durchhängenden Leitung.
    """

    print("\nAufgabe 9: Länge der Leitung")

    # Gesucht ist der Krümmungsradius a.
    # Gleichung:
    #  $x * \cosh(50 / x) - x - 10 = 0$ 
    funktion = "x * math.cosh(50 / x) - x - 10"

    # Sinnvolles Intervall für den Krümmungsradius
    a = 100
    b = 200

    ergebnis = bisektion(funktion, a, b, tol=1e-8)

    if ergebnis is not None:
        radius, iterationen = ergebnis

        # Abstand zwischen den Masten in Metern
        w = 100

        # Formel für die Leitungslänge
        laenge = 2 * radius * math.sinh(w / (2 * radius))

        print(f"Krümmungsradius a: {radius}")
        print(f"Iterationen: {iterationen}")
        print(f"Kontrolle: f(a) = {f(radius, funktion)}")
        print(f"Länge der Leitung: {laenge} m")

        print("\nGerundet:")
        print(f"Die Leitung ist ungefähr {round(laenge, 2)} m lang.")
```

Fehlerbehandlungen sind enthalten: Wenn im Intervall kein Vorzeichenwechsel vorhanden ist, gibt das Programm eine Fehlermeldung aus. Auch falsche Eingaben bei Funktionen oder Zahlen werden abgefangen.

Screenshot / Hauptmenü:

Fazit: Der Solver kann Nullstellen mit Bisektion und Regula falsi berechnen, die Genauigkeit testen, Ergebnisse visualisieren und ein reales Anwendungsproblem lösen.

```
def main() -> None:
    """Hauptprogramm mit Menü, damit jede Aufgabe einzeln testbar ist."""
    print("Nullstellenberechnung mit der Bisektionsmethode")
    print("1 = Aufgabe 5 (Bisektion)")
    print("2 = Aufgabe 8 (Polynom)")
    print("3 = Aufgabe 9 (Leitung)")

    try:
        auswahl = input("Wähle 1, 2 oder 3: ")

        if auswahl == "1":
            aufgabe5()

        elif auswahl == "2":
            aufgabe8()

        elif auswahl == "3":
            aufgabe9()

        else:
            print("Fehler: Bitte 1, 2 oder 3 eingeben.")

    except ValueError:
        print("Fehler: Bitte gültige Zahlen eingeben.")

if __name__ == "__main__":
    main()
```